

CO-215, Assignment 12

Roll number: CSB24056

Name: Rohit Bandhu

Session: Spring, Year: 2026

CODE :

.MODEL SMALL

.STACK 100H

.DATA

No_Records EQU 5

No_Fields EQU 7

Bytes_Record EQU No_Fields * 2

Subjects DB 3

; field1 = roll

; field2 = age

; field3-5 = marks ; field6 =

total

; field7 = average

Marks_Records DW 01,22,66,54,75,00,00

DW 02,21,70,67,77,00,00

DW 03,21,55,60,65,00,00

DW 04,22,75,66,83,00,00

DW 05,21,49,59,69,00,00

Output_Msg1 DB 'Roll No \$'

Output_Msg2 DB ': Total- \$'

Output_Msg3 DB ' Average- \$'

nwln DB 10,13,'\$'

ten DB 10 hun

DB 100

.CODE

main PROC

.STARTUP

```
;=====
; Calculate total and average
;=====
```

mov cx, No_Records xor bx, bx

Repeat1:

```
    xor ax, ax    add ax,
Marks_Records[bx+4]    add ax,
Marks_Records[bx+6]    add ax,
Marks_Records[bx+8]
```

```
    ; store total    mov Marks_Records[bx+10],
ax
```

```
    ; average = total / 3 (byte divisor: AX / Subjects -> AL=quot, AH=rem)    xor ah, ah    div Subjects
```

```
    ; store average (quotient only)    xor ah, ah
mov Marks_Records[bx+12], ax
```

```
    add bx, Bytes_Record
```

loop Repeat1

```
;=====;  
Print all records  
;=====
```

```
mov cx, No_Records xor bx,  
bx
```

Repeat2:

```
; newline before each record lea dx,  
nwln mov ah, 09h int 21h
```

```
; print "Roll No " lea dx,  
Output_Msg1 mov ah, 09h  
int 21h
```

```
; --- print roll number (2 digits) --- push bx push cx
```

```
mov ax, Marks_Records[bx] xor ah, ah ; clear AH so we divide AX by ten cleanly  
div ten ; AL = tens digit, AH = ones digit
```

```
mov cl, ah ; save ones digit in CL before AH is destroyed  
mov dl, al add dl, 30h mov ah, 02h int 21h  
; print tens digit
```

```
mov dl, cl ; retrieve ones digit add dl, 30h mov ah,  
02h int 21h ; print ones digit  
pop cx  
pop bx
```

```
; print ": Total- " lea dx,  
Output_Msg2 mov ah, 09h  
int 21h
```

```
; --- print total (3 digits) --- push bx push cx
```

```
mov ax, Marks_Records[bx+10] xor ah, ah div hun ; AL = hundreds digit, AH =  
remainder (0-99)
```

```
mov cl, ah ; save remainder in CL  
mov dl, al add dl, 30h mov ah, 02h int 21h ;  
print hundreds digit
```

```
mov al, cl ; remainder back into AL xor ah, ah div ten ; AL =  
tens digit, AH = ones digit
```

```
mov cl, ah ; save ones digit  
mov dl, al add dl, 30h mov ah, 02h int 21h  
; print tens digit  
mov dl, cl add dl, 30h mov ah, 02h int 21h  
; print ones digit  
pop cx  
pop bx
```

```

    ; print " Average- "    lea dx,
Output_Msg3    mov ah, 09h
int 21h

    ; --- print average (2 digits) ---    push bx    push
cx
    mov ax, Marks_Records[bx+12]    xor ah, ah    div ten    ; AL = tens digit,
AH = ones digit

    mov cl, ah    ; save ones digit in CL
    mov dl, al    add dl, 30h    mov ah, 02h    int 21h
; print tens digit
    mov dl, cl    add dl, 30h    mov ah, 02h    int 21h
; print ones digit
    pop cx
pop bx

    ; advance to next record    add bx,
Bytes_Record
    dec cx    jz
ExitLoop

```

```

    jmp Repeat2

ExitLoop:

mov ah, 4Ch mov al,
00h
int 21h

main ENDP END
main

```

Observations:

Variables:

Name	No of Bytes	Location	Initial Value	Step number which value does not change	Final value
No_Records	2	N/A	5	Never changes	5
No_Fields	2	N/A	7	Never changes	7
Byte_Record	14	N/A	14	Never changes	14
Marks_Records	70	008B	01, 22, 66, 54, 75, 00, 00 02, 21, 70, 67, 77, 00, 00 03, 21, 55, 60, 65, 00, 00 04, 22, 75, 66, 83, 00, 00 05, 21, 49, 59, 69, 00, 00	5th Iteration of Repeat1 loop	01, 22, 66, 54, 75, 195, 65 02, 21, 70, 67, 77, 214, 71 03, 21, 55, 60, 65, 180, 60 04, 22, 75, 66, 83, 224, 74 05, 21, 49, 59, 69, 177, 59
Subject_Av	14	00D1	Undefined/Garbage (?)	Never changed as it is never used in code	Undefined/Garbage (?)

Learning Outcome:

1. Instruction structures in μP 8086 ALP for based-indexed addressing: In based-indexed addressing mode, a base register (BX or BP) is used along with a constant displacement value to access a specific memory location. The processor adds the value in the base register to the displacement to calculate the final address of the data. In this program, BX holds the starting offset of the current record in the array, and the displacement (like +4, +10, +12) selects which field inside that record we want to access. For example, Marks_Records[bx+4] means go to the address of Marks_Records, then add BX and 4 to reach the third field. After processing each record, BX is increased by 14 (Bytes_Record) so that it now points to the next record. This way we can loop through all records just by changing BX without having to hardcode separate addresses for each record.
2. Use of various registers in the division instruction operation in μP 8086: The 8086 processor uses specific registers for the DIV instruction depending on whether the divisor is a byte or a word. When dividing by a byte, the dividend has to be placed in the AX register before the division. After the DIV instruction executes, the quotient goes into AL and the remainder goes into AH. In this program, since Subjects, ten, and hun are all byte variables, byte division is used throughout. Before each division, AH is cleared using xor ah, ah so that there is no leftover garbage value in AH that could mess up the result. One important thing we learned is that after the division, the remainder sitting in AH gets destroyed when we

set AH = 02h for the DOS print interrupt. So we have to save the remainder into another register like CL before printing, otherwise the wrong digit gets printed. This was actually the main bug in the original code that was causing wrong output.

Q.1: Write an 8086 Assembly Language Program to create a 2×2 mul dimensional array, store data in the array, and display all the elements one by one on the screen.

```
.MODEL SMALL
.STACK 100H

.DATA

    Arr DB 1,2
         DB 3,4
.CODE
MAIN PROC
.STARTUP
    MOV CX,4
    MOV SI,0
AGAIN:
    MOV DL,Arr[SI]
    ADD DL,30H
    MOV AH,02H
    INT 21H

    MOV DL,''
    MOV AH,02H
    INT 21H

    INC SI

    LOOP AGAIN
    MOV AH,4CH
    INT 21H

MAIN ENDP
END MAIN
```

Q.2: Write an 8086 Assembly Language Program to create a 2×2 mul dimensional array, take array elements as input from the keyboard, store them in the array, and display all the elements one by one on the screen.

```
.MODEL SMALL
.STACK 100H

.DATA

Arr DB 4 DUP(?)

Msg1 DB 'Enter 4 elements: $'
Msg2 DB 10,13,'Array Elements: $'

.CODE
MAIN PROC
.STARTUP
```

```
; -----  
; DISPLAY INPUT MESSAGE  
; -----  
LEA DX,Msg1    MOV AH,09H  
INT 21H
```

```

; ----- ; TAKE INPUT FROM
USER
; -----
MOV CX,4
MOV SI,0
INPUT:
; Read character from keyboard
MOV AH,01H
INT 21H

; Convert ASCII to number
SUB AL,30H

; Store in array
MOV Arr[SI],AL
INC SI
LOOP INPUT

; ----- ; DISPLAY OUTPUT
MESSAGE
; -----
LEA DX,Msg2
MOV AH,09H
INT 21H

; -----
; DISPLAY ARRAY ELEMENTS
; -----
MOV CX,4
MOV SI,0
DISPLAY:
; Load array value
MOV DL,Arr[SI]

; Convert number to ASCII
ADD DL,30H

; Print character
MOV AH,02H
INT 21H

; Print space
MOV DL,' '
MOV AH,02H
INT 21H

INC SI

LOOP DISPLAY

; ----- ; EXIT PROGRAM
; -----

MOV AH,4CH
INT 21H

```


MAIN ENDP
END MAIN

Q.3: Write an 8086 Assembly Language Program to create a 5×7 mul dimensional array, store data in the array, and display all the elements one by one on the screen.

.MODEL SMALL

.STACK 100H

.DATA

Arr DB 1,2,0,5,3,6,7

DB 9,1,6,3,5,0,9

DB 1,3,4,0,8,0,7

DB 0,2,3,9,5,5,4

DB 1,5,9,4,2,6,1

Msg1 DB '5 x 7 Array Elements: \$' nwl n DB 10,13,'\$'

.CODE

MAIN PROC

.STARTUP

LEA DX,Msg1

MOV AH,09H

INT 21H

LEA DX,nwl n

MOV AH,09H

INT 21H

MOV BX,5

MOV SI,0

ROW_LOOP:

MOV CX,7

COL_LOOP:

MOV DL,Arr[SI]

ADD DL,30H

MOV AH,02H

INT 21H

MOV DL,''

MOV AH,02H

INT 21H

INC SI

LOOP COL_LOOP

LEA DX,nwl n

MOV AH,09H

INT 21H

DEC BX

JZ DONE

JMP ROW_LOOP

DONE:

MOV AH,4CH

INT 21H

MAIN ENDP

END MAIN

Q.4: Write an 8086 Assembly Language Program to create a 5×7 multidimensional array, store data in the array through keyboard input, and display all the elements one by one on the screen.

```
.MODEL SMALL
.STACK 100H

.DATA

Arr DB 35 DUP(?)

Msg1 DB 'Enter 35 elements (0-9):', 13, 10, '$' Msg2 DB 10,13,'Array
Elements:', 13, 10, '$' nwln DB 10,13,'$'

.CODE
MAIN PROC
.STARTUP

; -----
; DISPLAY INPUT MESSAGE
; -----
LEA DX,Msg1
MOV AH,09H
INT 21H

; ----- ; TAKE INPUT FROM
USER
; -----
MOV BX,5
MOV SI,0

INPUT_ROW:
MOV CX,7

INPUT_COL:
MOV AH,01H
INT 21H

SUB AL,30H
MOV Arr[SI],AL INC SI
LOOP INPUT_COL

LEA DX,nwln
MOV AH,09H
INT 21H

DEC BX
JZ INPUT_DONE JMP
INPUT_ROW

INPUT_DONE:
; ----- ; DISPLAY OUTPUT
MESSAGE
; -----
LEA DX,Msg2
MOV AH,09H
```

INT 21H

```

; ----- ; DISPLAY ARRAY
ELEMENTS
; -----
```

```
    MOV BX,5
    MOV SI,0
DISPLAY_ROW:
    MOV CX,7
DISPLAY_COL:
    MOV DL,Arr[SI]
    ADD DL,30H
    MOV AH,02H
    INT 21H

    MOV DL,''
    MOV AH,02H
    INT 21H

    INC SI
    LOOP DISPLAY_COL

    LEA DX,nwln
    MOV AH,09H
    INT 21H

    DEC BX
    JZ DONE
    JMP DISPLAY_ROW

DONE:
    MOV AH,4CH
    INT 21H

MAIN ENDP
END MAIN
```